

Placement Portal Application – Project Report

Author

Name: Deepesh Dey

Roll Number: 24f3002746

Email: 24f3002746@ds.study.iitm.ac.in

I am currently pursuing an Integrated MSc in Mathematics and Computation from BIT Mesra and enrolled in IIT Madras BS Degree in Data Science.

Project Details

Project Title: Placement Portal Application

Project Name: CareerRoot

Project Statement: Institutes require efficient systems to manage campus recruitment activities involving students, companies, and administrators. Traditional methods such as spreadsheets and emails often lead to inefficiencies in tracking applications and approvals. This project develops a Placement Portal Application that centralizes the placement process with role-based access for Admin, Companies, and Students. The system is built using Flask, Jinja2, Bootstrap, and SQLite without using JavaScript for core logic.

Project Structure

```
PlacementPortal_24f3002746/
├── app.py                # Flask application entry point
├── careerroot.db         # SQLite database
├── requirements.txt      # Dependencies
├── core/
│   ├── database/
│   │   ├── models.py    # ORM models (Admin, Company, Student, PlacementDrive, Application, Placement)
│   │   └── forms.py
│   ├── routes/
│   │   ├── auth.py      # Login/logout
│   │   ├── admin.py     # Admin dashboard
│   │   ├── company.py   # Company portal
│   │   ├── student.py   # Student portal
│   │   └── api.py
│   ├── utils/
│   │   ├── validators.py # Input validation
│   │   └── chart_data.py # Analytics
│   └── auth.py          # Auth utilities
├── static/              # CSS, JS, images
├── templates/           # HTML templates (Jinja2)
└── .git/                # Version control
```

AI/LLM Usage

Backend (Flask + DB): 20%

Frontend (HTML/JS): 15%

Auth + CRUD Logic: 12%

Styling/UI: 8%

Extras: 9%

I used ChatGPT (GPT-5) and Claude Haiku 4.5 to assist in basic code structuring, template generation, and improving naming consistency. The extent of AI/LLM usage is around 15–20%, limited to code suggestions and documentation formatting. All final implementation logic, debugging, and integration were done manually.

Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	ORM for SQLite database
Jinja2	Template engine for dynamic pages
Bootstrap 5	UI styling and responsiveness
Flask-Login	Authentication and session management
WTForms	Form handling and validation
SQLite	Database for storing application data

DB Schema Design: The database is designed using a relational structure with the following core tables:

- **Admin:** Stores administrator details such as admin_id (Primary Key), username, email (unique), and password.
- **Company:** Contains company information including company_id (Primary Key), company_name, email (unique), password, and approval_status.
- **Student:** Stores student data such as student_id (Primary Key), name, email (unique), department, cgpa, and placement_status.
- **PlacementDrive:** Includes details of job postings such as drive_id (Primary Key), company_id (Foreign Key), job_title, eligibility_criteria, salary_range, application_deadline, and status.
- **Application:** Tracks student applications with application_id (Primary Key), student_id (Foreign Key), drive_id (Foreign Key), and status.
- **Placement:** Records final placements with placement_id (Primary Key), student_id (Foreign Key), company_id (Foreign Key), and salary_package.

Constraints and Design Reasoning:

- Primary and foreign keys are used to maintain relationships between tables.
- Unique constraints on email fields prevent duplicate registrations.
- The schema follows normalization principles to reduce redundancy.
- Status fields are used to track workflow stages such as approval and application progress.

API Design

The application implements role-based APIs using Flask routes to handle interactions between different users.

- **Authentication APIs:** Handle login and logout for Admin, Company, and Student.
- **Admin APIs:** Used for approving companies and placement drives, and managing users.
- **Company APIs:** Allow companies to create placement drives, view applications, and select candidates.
- **Student APIs:** Enable students to view available jobs, apply for drives, and track application status.

Implementation:

APIs are implemented using Flask routing with session-based authentication via Flask-Login. Role-based access control ensures that each user can only access permitted functionalities. The API structure follows a modular approach, and the YAML file defining endpoints is submitted separately.

Architecture and Features The project follows an MVC-like architecture for better organization and maintainability.

The **models** are defined using SQLAlchemy in the database module, the **controllers** are implemented using Flask routes organized into separate files for admin, company, student, and authentication, and the **views** are handled using Jinja2 templates stored in the templates folder. Static files such as CSS are managed in the static directory.

Features Implemented:

- Multi-role authentication system for Admin, Company, and Student
- Company registration and approval workflow
- Placement drive creation and management
- Student job application and tracking system
- Admin dashboard for monitoring activities.

Additional Features:

- Role-based access control for security
- Status tracking for applications and approvals
- Responsive UI using Bootstrap

Conclusion

The system simplifies placement management by improving efficiency and reducing manual work while demonstrating full-stack development concepts.

Video Presentation

Drive Link:

<https://drive.google.com/file/d/1s-OXMJ1l4lgpznnUmo5KDXZRkLKPDxc1/view?usp=sharing>

(Accessible to all with "View" permission.)

